

# Datenanalyse in der Physik

Marcel Lüthi (D-INFK)

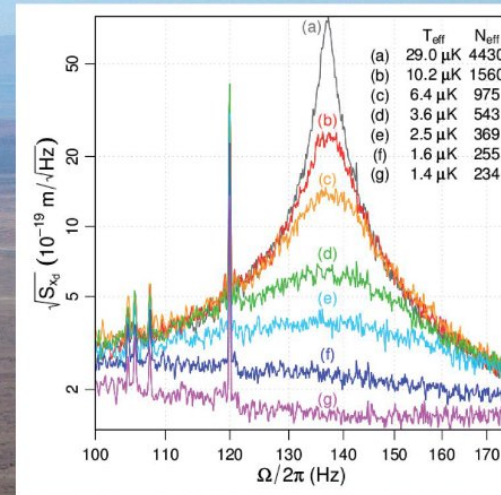
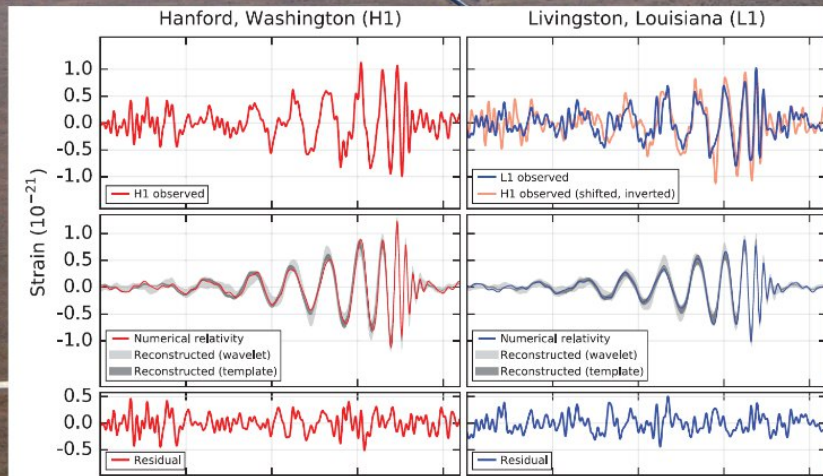
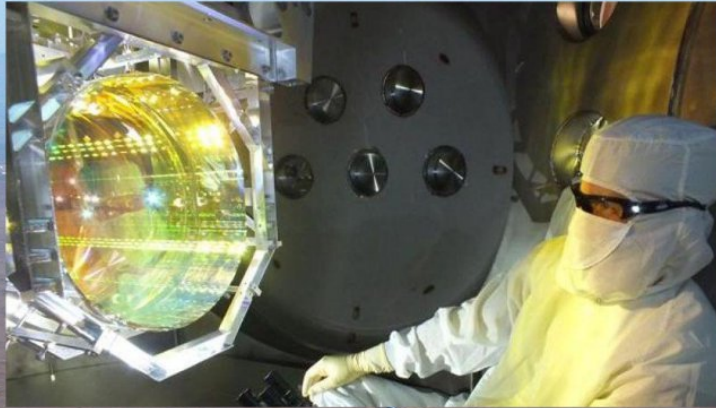
Vorlesung am D-Phys der ETH Zürich





# Einführung erste Woche

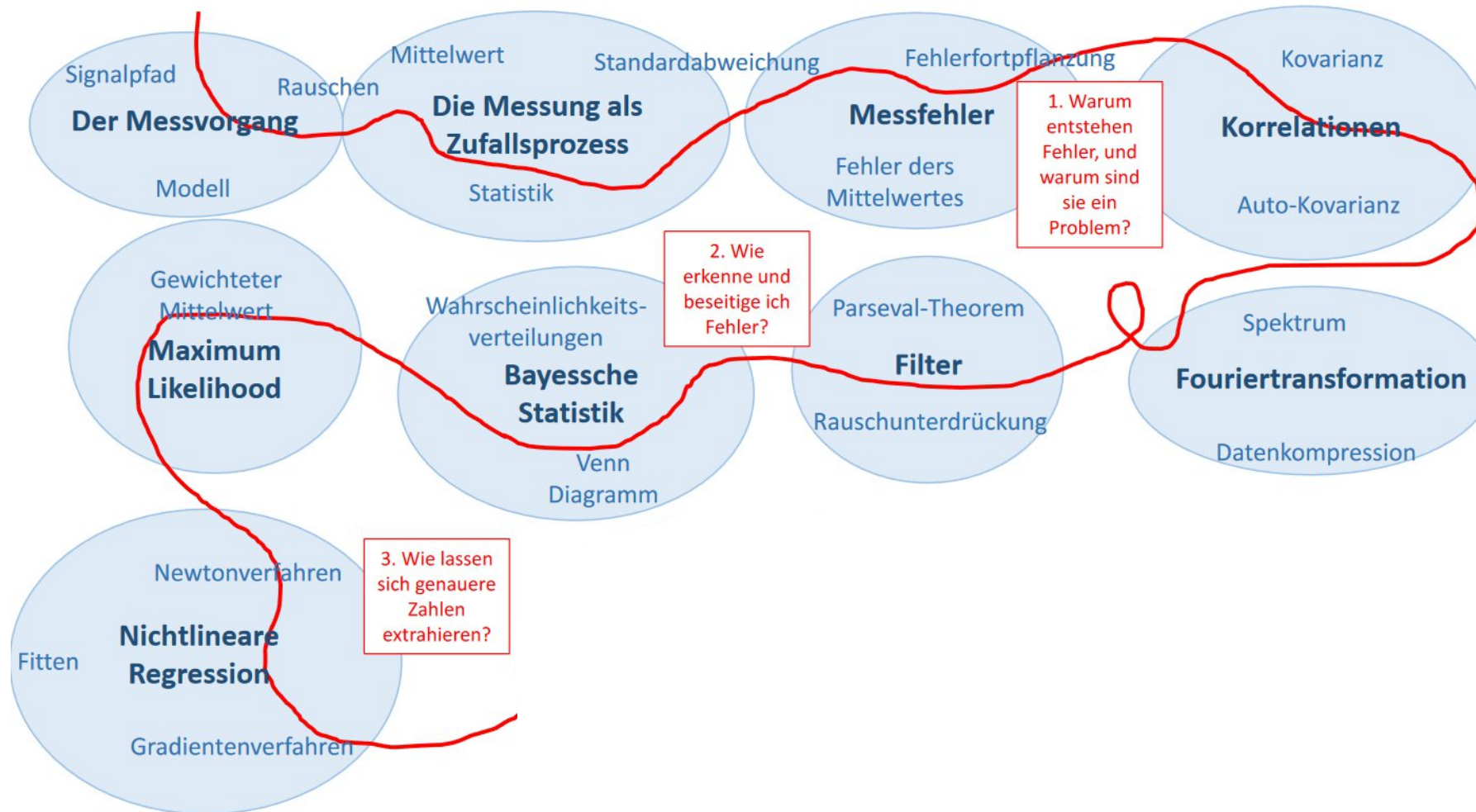
LIGO



B. Abbott *et al.*, New J. Phys. 11, 073032 (2009)



# Der rote Faden



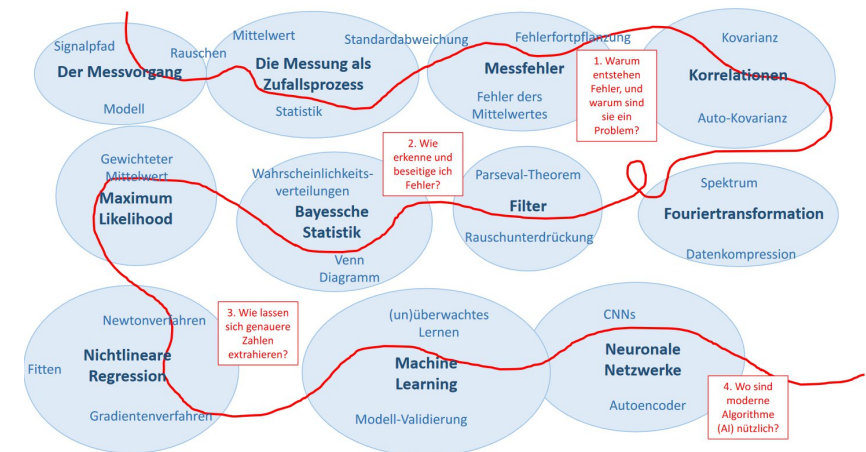
# Diskussion

## Definition: Maschinelles Lernen (T. Mitchell)

A computer program is said to learn from **experience E** with respect to some class of **tasks T** and **performance measure P**, if its performance at tasks in T, as measured by P, improves with experience E

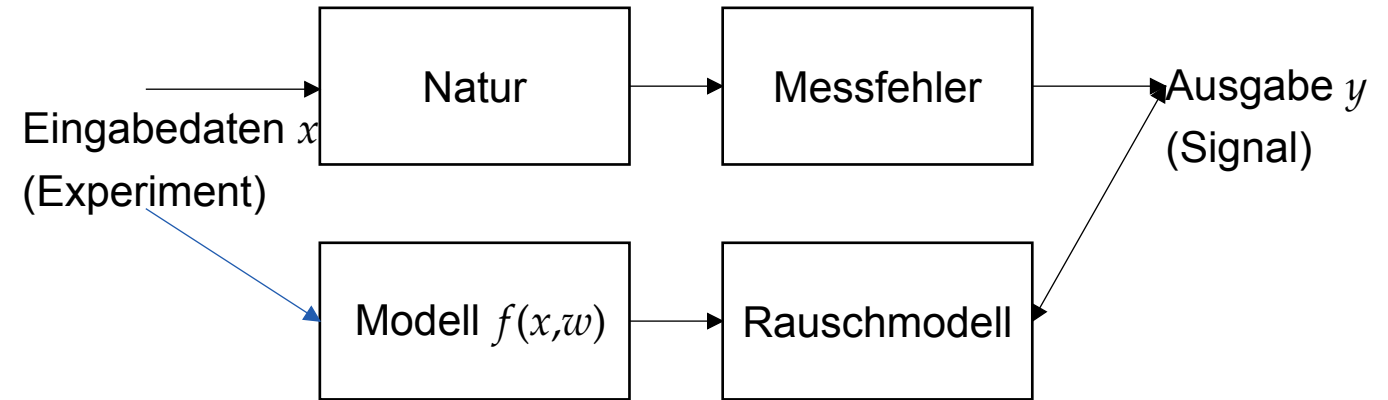
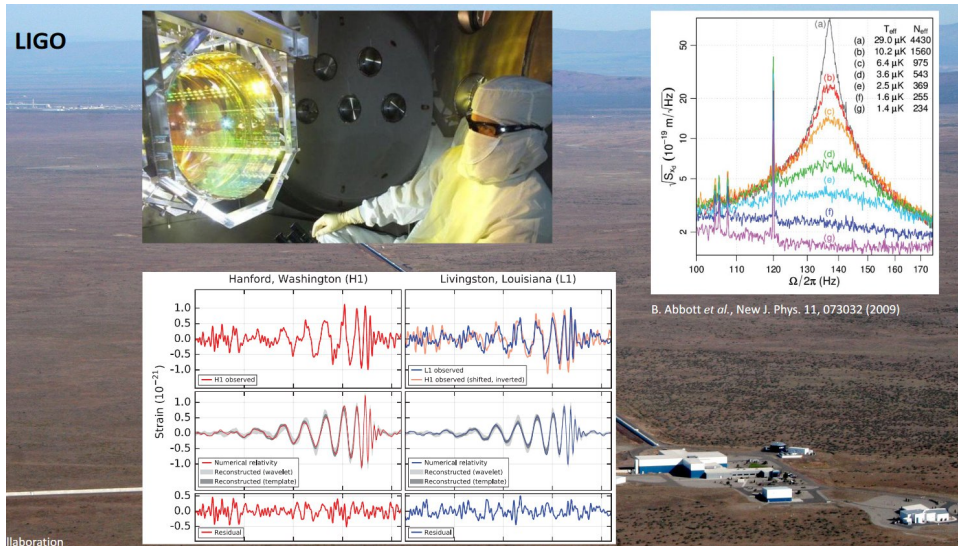
Diskutieren sie folgende Fragen:

- Welche Methoden, die Sie bisher gelernt haben, könnten als *Maschinelles Lernen* bezeichnet werden?
- Was ist Ihr primäres Ziel, wenn Sie als Physiker:in diese Methoden einsetzen? Werden im Maschinellen Lernen (gemäss obiger Definition) dieselben Ziele verfolgt?





# Datenanalyse in der Physik



## Gegeben

- Mathematisches Modell (Hypothese / Theorie)
- Daten eines Experiments

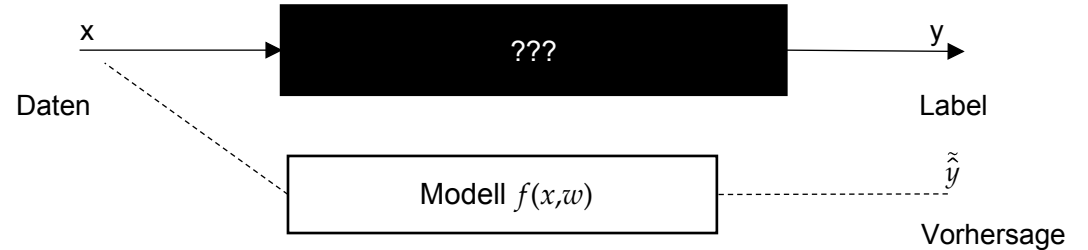
## Aufgabe

- Verifizieren des Modells mit den Daten aus den Experimenten

# Maschinelles Lernen – Gleich und doch anders



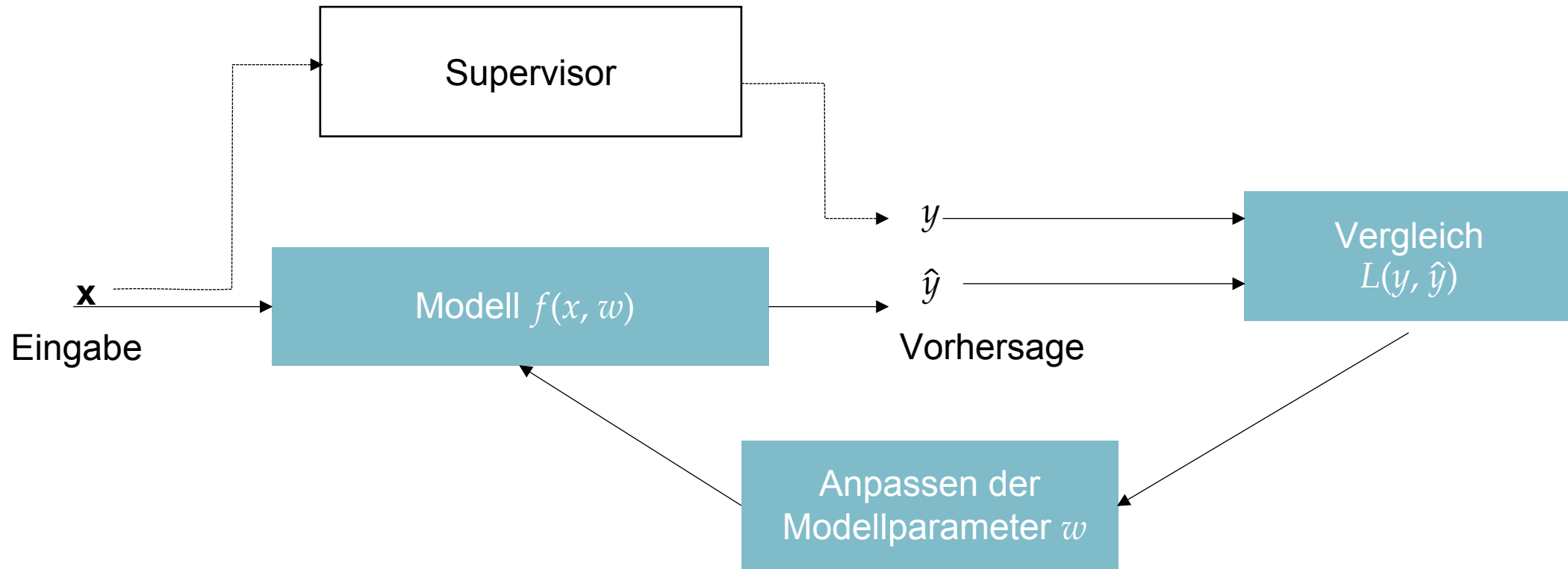
- Regisseur: Tom Fox
- Länge: 93 Minuten
- Darsteller: Eichhörnchen, Hasen, Füchse
- FSK: 14
- ...



- **Gegeben** Daten (Messwerte / Beobachtungen) – Oft nicht systematisch gesammelt
- **Aufgabe** Modell finden, welches neue Vorhersagen machen kann
  - Physikalischer Zusammenhang oft nicht gegeben oder nicht von primären Interesse

# Überwachtes Lernen (supervised learning)

Eingaben  $x \in \mathbb{R}^d$



# Zwei Kategorien von Problemen: Regression und Klassifikation

## Gegeben:

- Datenpaare  $\{(x_1, y_1), \dots, (x_n, y_n)\}$  bestehend aus Merkmalen  $x_i$  und Zielvariable  $y_i$

**Regression:** Zielvariable  $y$  variiert kontinuierlich ( $y \in \mathbb{R}$ )

**Klassifikation:** Zielvariable  $y$  kann nur diskrete Werte annehmen ( $y \in \{0, \dots, k\}$ )

## Beispiel (Film)

- Merkmale  $x$ : Länge des Films, Anzahl Dialoge im Film, Anzahl Stars die Mitwirken
- Regression: Zielvariable  $y$  zum Beispiel eingespielter Gewinn
- Klassifikation: Zielvariable  $y$  zum Beispiel Kritikerbewertung (1-5 Sterne)



# Agenda der letzten 3 Vorlesungen

## **Heute:**

- Grundprinzipien des Maschinellen Lernen
- Regression als Machine Learning Modell
- Trainieren von Machine Learning Modellen
- Einführung in Scikit-Learn

## **Nächste Woche:** Neuronale Netze / Deep Learning

- Klassifikation und Logistische Regression
- Einführung in neuronale Netze

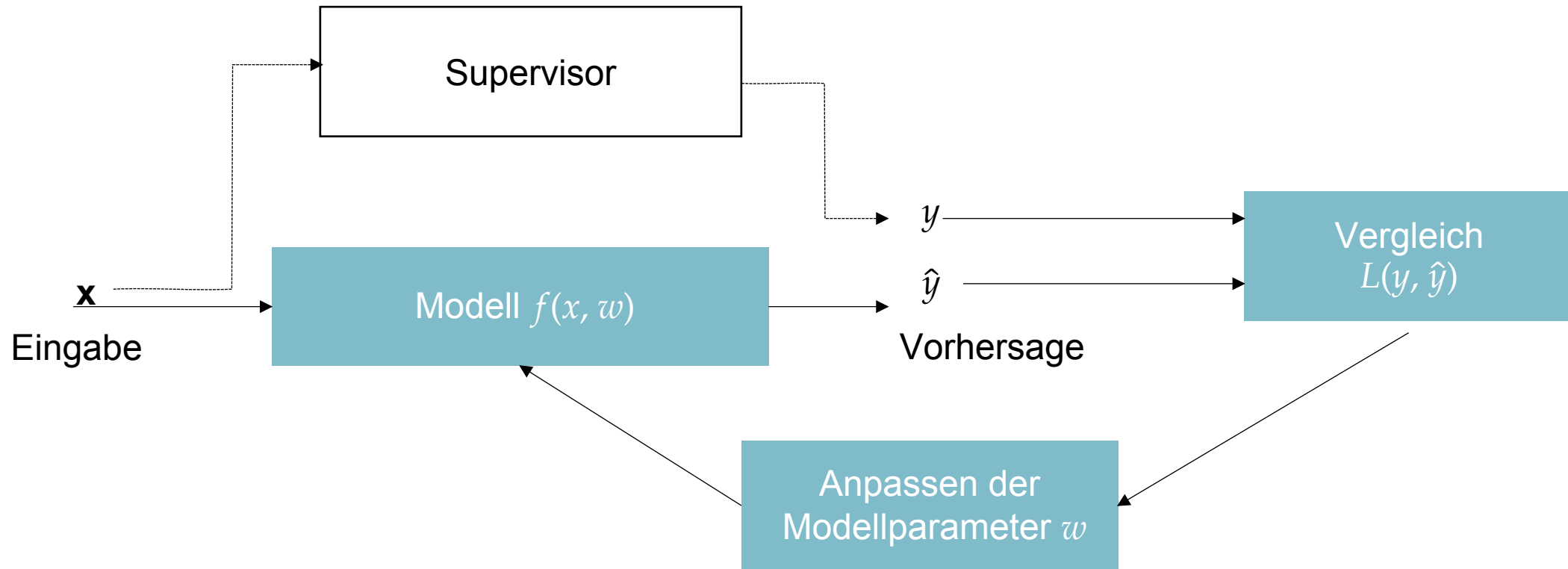
## **Übernächste Woche:** Weitere Themen im Maschinellen Lernen

- Encoding von nicht-numerischen Daten
- Clustering
- Dimensionalitätsreduktion

# Regression

# Überwachtes Lernen (supervised learning)

Eingaben  $x \in \mathbb{R}^d$





# Einfachstes Beispiel: Lineare Regression

## Daten

$$D = \{(x_1, y_1), \dots, (x_n, y_n)\}, x_i \in \mathbb{R}, y_i \in \mathbb{R}$$

## Modell

$$\hat{y} = f(x, w) = f(x, w_1, w_0) = w_1 x + w_0$$

## Parameter

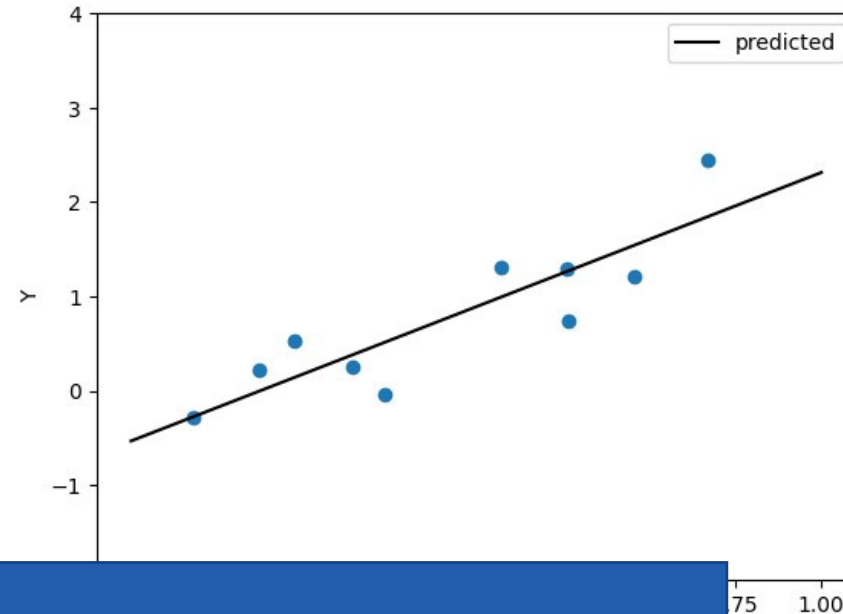
- Parameter vector  $w = (w_0, w_1)^T$
- Steigung  $w_1$ , Achsenabschnitt  $w_0$

## Verlustfunktion

$$L(y, \hat{y}) = (y - f(x, w))^2$$

## Anpassen der Parameter

- Optimierungsproblem  $\min_w \sum_{i=1}^n (y_i - f(x_i, w))^2$   
(Lösung durch Normalgleichung)



Achtung neue Notation:  
Modellparameter  $w$  (weights)  
statt  $a, b$

Also  $f(x, w_1, w_0) = w_1 x + w_0$   
statt  $f(x, a, b) = ax + b$

# Ein Detail: Standardisierung von Daten

Viele Algorithmen im Maschinellen Lernen funktionieren am besten mit dimensionslosen, standardisierten Daten.

- Bessere Konvergenz bei gradientenbasierter Optimierung
- Koeffizienten (Gewichte) werden vergleichbarer

## Standardisierung eines Merkmals $x$

1. Mittelwert  $\mu$  abziehen (Position wird irrelevant)
2. Durch Standardabweichung  $\sigma$  teilen  
(Skala und Einheit fallen weg)

$$z = \frac{x - \mu}{\sigma}$$

## Rücktransformation

$$x = \sigma z + \mu$$

# Trainieren eines Regressionsmodells mit Scikit-learn

## **scikit-learn:**

- Populäre Machine learning Bibliothek in Python
- Quasi standard in Data Science
- Effiziente Implementation von standard Lernalgorithmen hinter wohldefiniertem Interface

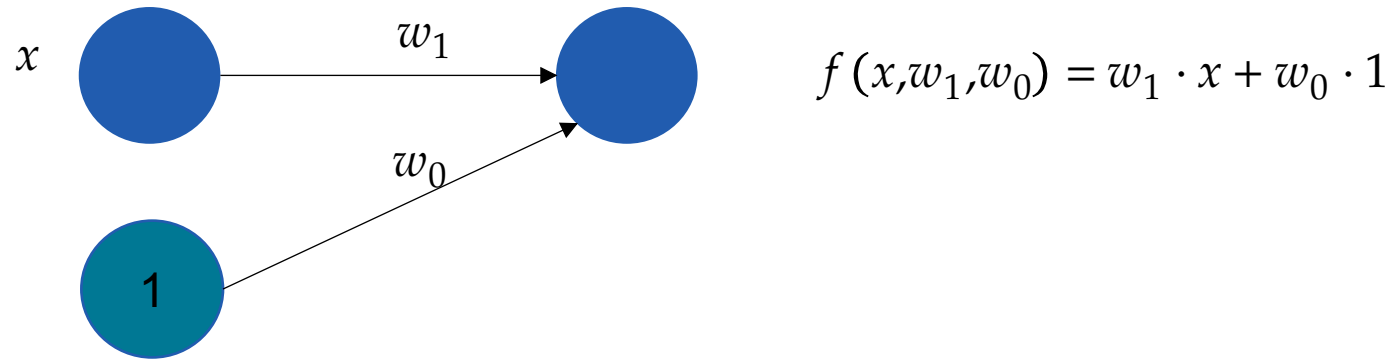
```
# Skalieren der Merkmale
scaler = StandardScaler()
scaler.fit(X)
X_scaled = scaler.transform(X)

# Erstellen des Modells
model = LinearRegression()

# Anpassen (trainieren) des Modells
model.fit(X_scaled, y)

# Vorhersagen machen
y_pred = model.predict(X_scaled)
```

# Lineare Regression: Grafisches Modell





# Lineare Regression in höheren Dimensionen

## Daten

$$D = \{(x_1, y_1), \dots, (x_n, y_n)\}, x_i \in \mathbb{R}^d, y_i \in \mathbb{R}$$

## Modell

$$\hat{y} = f(x, w) = w_1 x_1 + \dots + w_d x_d + w_0$$

## Parameter:

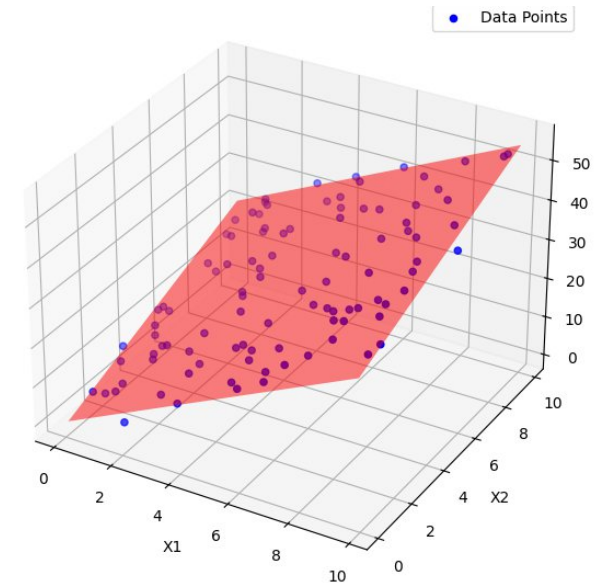
- $w = (w_0, w_1, w_2, \dots, w_d)^T$
- Gewicht  $w_i$  bestimmt Einfluss von Komponente  $x_i$

## Verlustfunktion

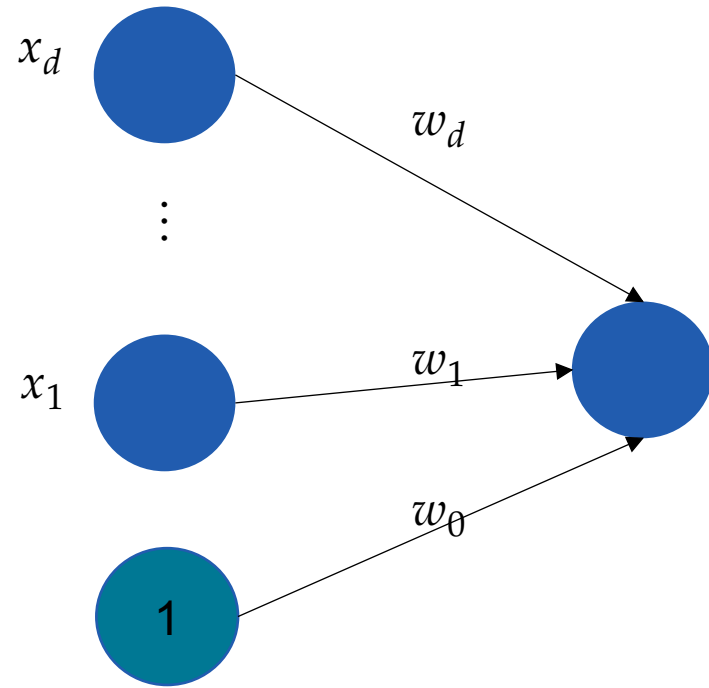
$$L(y, \hat{y}) = (y - f(x, w))^2$$

## Anpassen der Parameter

- Optimierungsproblem  $\min_w \sum_{i=1}^n (y_i - f(x_i, w))^2$  (Lösung durch Normalgleichung)



# Lineare Regression in höheren Dimensionen: Grafisches Modell



$$f(x, w) = \sum_{i=1}^d w_i x_i + w_0 \cdot 1$$

# Polynomielle Regression

## Daten

$$D = \{(x_1, y_1), \dots, (x_n, y_n)\}, x_i \in \mathbb{R}, y_i \in \mathbb{R}$$

## Polynome

$$\varphi_1(x) = x^1, \dots, \varphi_m(x) = x^m$$

## Modell

$$f(x, \mathbf{w}) = f(x, w_0, \dots, w_m) = \sum_{i=1}^m w_i \varphi_i(x) + w_0$$

## Parameter

- Gewichte  $\mathbf{w} = (w_0, w_1, w_2, \dots, w_m)^T$

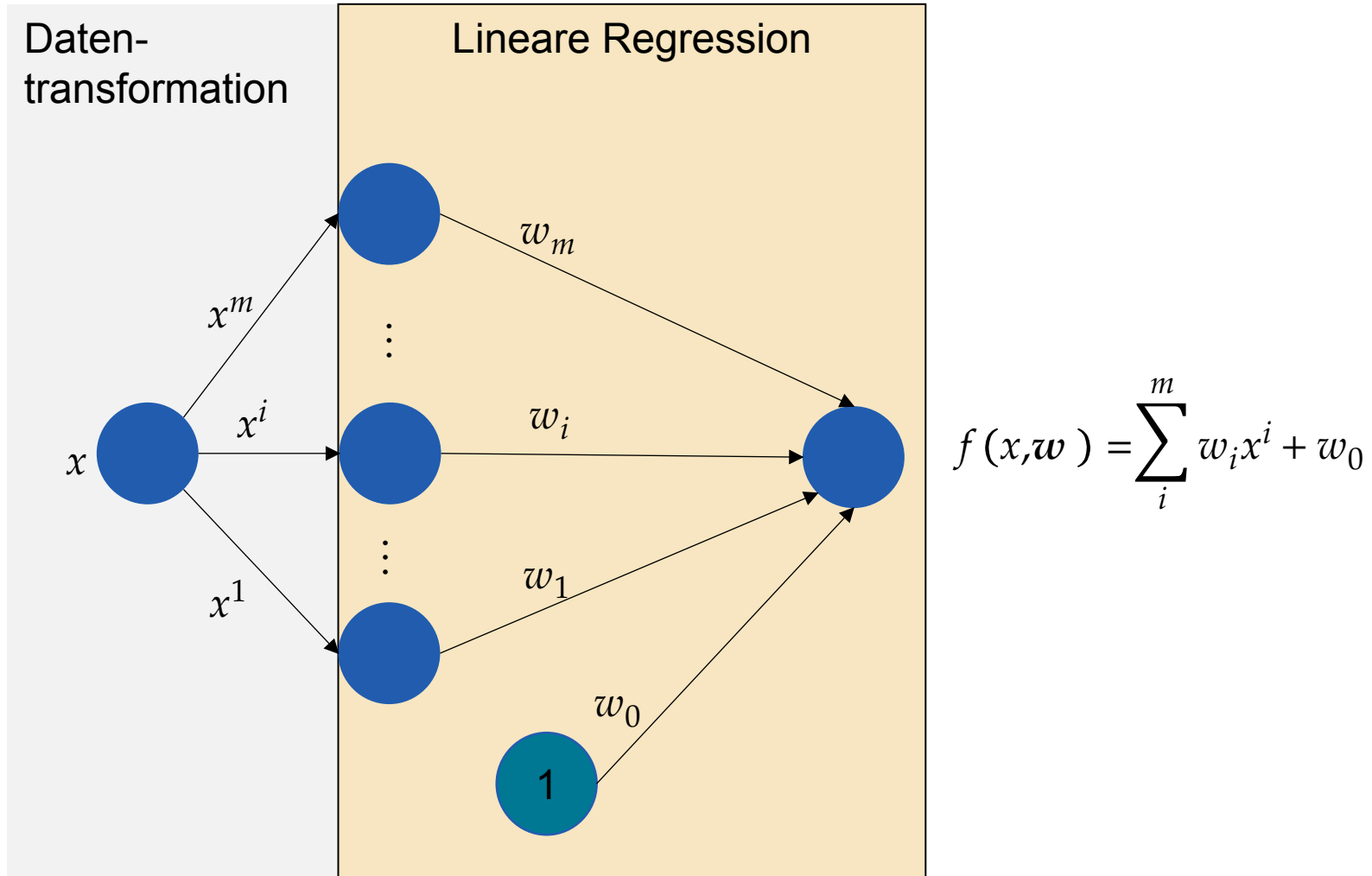
## Verlustfunktion

$$L(y, \hat{y}) = (y - f(x, \mathbf{w}))^2$$

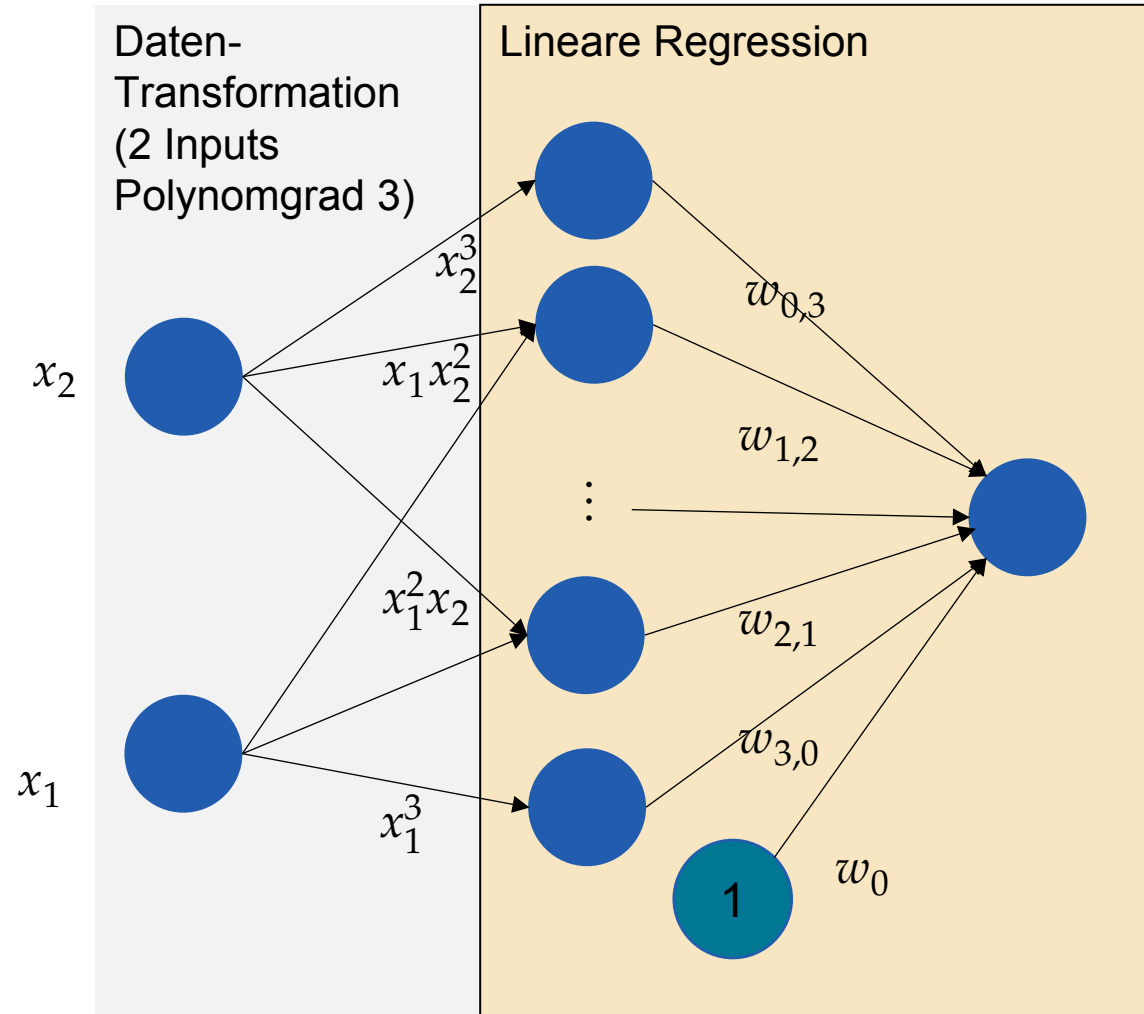
## Anpassen der Parameter

- Optimierungsproblems  $\min_{\mathbf{w}} \sum_{i=1}^n (y_i - f(x_i, \mathbf{w}))^2$  ((Lösung durch Normalgleichung))

# Polynomielle Regression: Grafisches Modell



# Polynomielle Regression mit mehreren Inputs: Grafisches Modell



$$f(x, w) = \sum_{k_1 + \dots + k_d \leq m} w_{k_1, \dots, k_d} x_1^{k_1} \dots x_d^{k_d}$$

# Trainieren eines Regressionsmodells mit polynomialen Features

```
from sklearn.linear_model import LinearRegression  
from sklearn.preprocessing import PolynomialFeatures
```

```
# Transformieren der Daten
```

```
poly = PolynomialFeatures(degree=3)  
X_poly = poly.fit_transform(X)
```

```
# Erstellen und trainieren des Regressionsmodells
```

```
poly_model = LinearRegression()  
poly_model.fit(X_poly, y)
```

```
# Vorhersagen von Datenpunkten
```

```
y_poly_pred = poly_model.predict(X_poly)
```



# Regression in Scikit-Learn



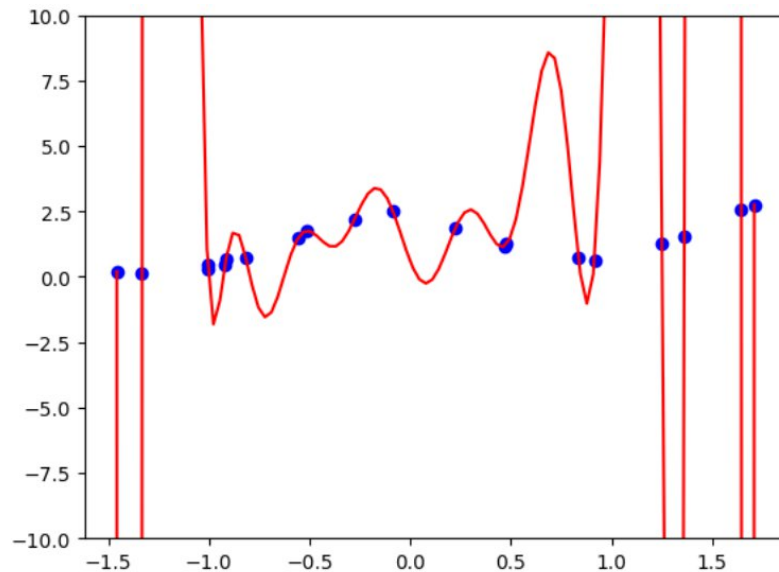
`12-lecture-notebook.ipynb`

# Overfitting

Modell lernt Trainingsdaten auswendig (inklusive Rauschen), statt Muster zu lernen

## Folgen

- Schlechte Vorhersagen auf unbekannten Daten.
- Modell wird sehr kompliziert
  - Sehr grosse Parameterwerte



```
Polynomial Coefficients: [[ 0.00000000e+00 -8.42768583e+00 -2.38143655e+01  2.92034433e+02  
 9.41827930e+02 -5.37918077e+03 -1.18488231e+04  4.49959167e+04  
 7.04016784e+04 -1.94458005e+05 -2.12607352e+05  4.47357159e+05  
 3.10323301e+05 -5.21082027e+05 -1.71640392e+05  2.42768282e+05]]
```

# Variation von Linearer Regression: Ridge regression

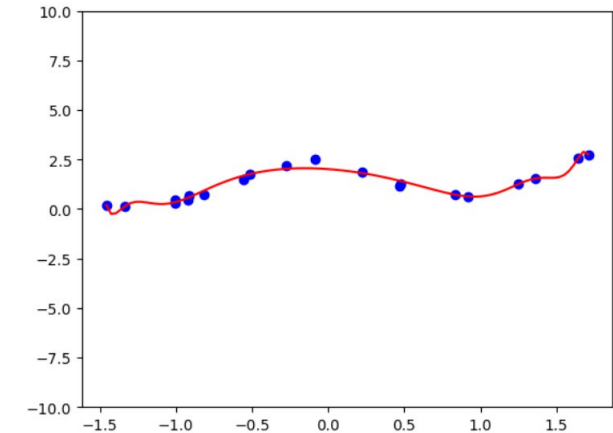
**Beobachtung:** Kleine Koeffizienten führen zu “einfacheren” Lösungen

## Ridge regression

- “Einfache” Lösungen durch Bestrafung grosser Parameterwerte  
"regularisieren"

$$\min_w \frac{1}{n} \sum_{i=1}^n (y_i - f(x, w))^2 + \alpha \|w\|^2$$

- Regularisierungsparameter  $\alpha \in \mathbb{R}^+$  bestimmt wie stark *regularisiert* wird



```
from sklearn.linear_model import Ridge

model = Ridge(alpha=0.1)
model.fit(X, y)
y_pred = model.predict(X)
```

# Overfitting und Regularisierung



`12-lecture-notebook.ipynb`

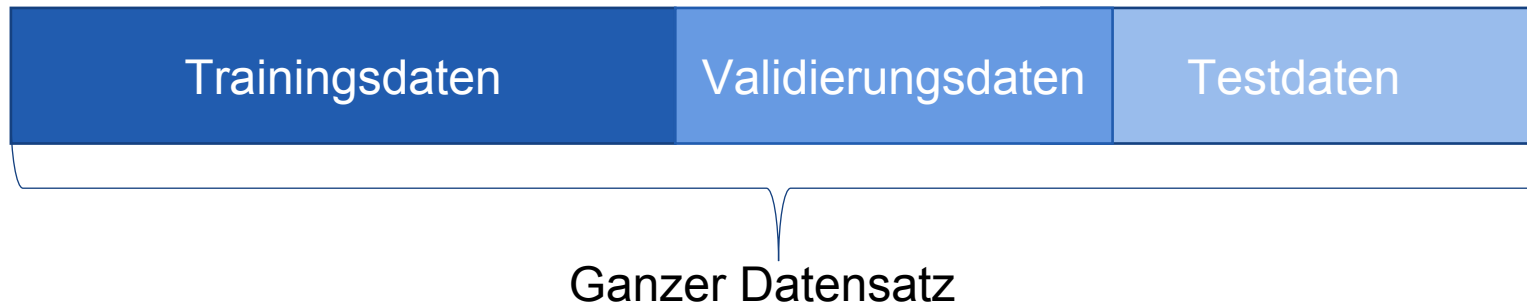
# Trainieren eines machine learning Modells

# Diskussion

- Können Sie eine Strategie finden, womit Sie die erkennen können, ob ein Modell gut gelernt hat oder overfitted?
  - Hinweis: Grafische Methoden wie in unserem 1D Beispiel sind im allgemeinen nicht verfügbar
  - Tip: Denken Sie an ihr eigenes Lernen und wie Ihr Lernen getestet wird.
- Können Sie sich ein einfaches Verfahren ausdenken, um gute Werte für den Polynomgrad und den Regularisierungsparameter zu finden?

# Terminologie: Trainings- und Testdaten

1. Trainingsdaten: Darauf wird Modell gelernt (trainiert).
2. Validierungsdaten: Werden verwendet um Hyperparameter zu finden und Lernfortschritt abzuschätzen
3. Testdaten: Darauf wird Vorhersagegenauigkeit evaluiert:  
Wichtig: Testdaten dürfen in keiner Phase im Training verwendet werden



```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```



# Terminologie: Parameter vs. Hyperparameter

**Parameter:** Wird während Training aus Daten "gelernt"

**Beispiel:** Koeffizienten  $w$  in (Ridge) Regression

**Hyperparameter:** Werden von Benutzer vor dem Training festgelegt

**Beispiel:** Regularisierungsparameter  $\alpha$  in Ridge Regression

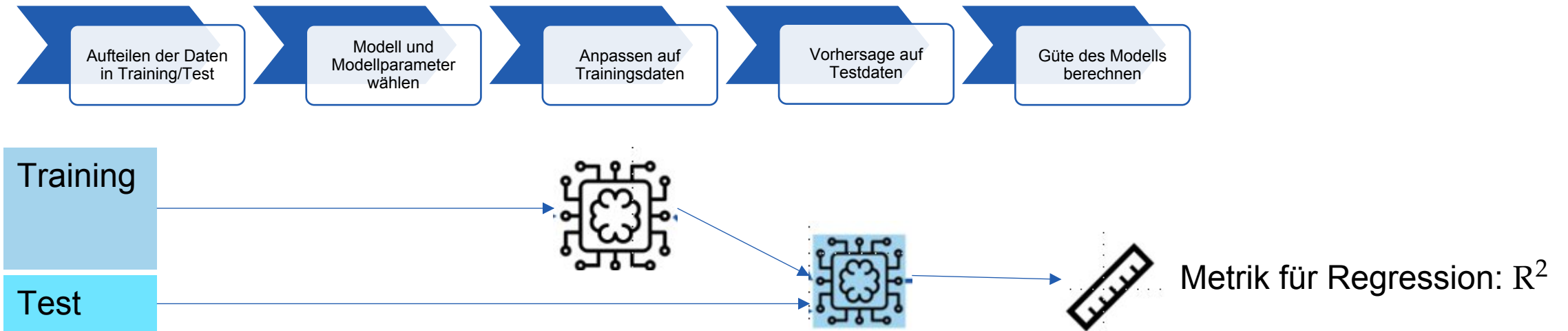
Ridge regression

$$\min_w \frac{1}{n} \sum_{i=1}^n (y_i - f(x, w))^2 + \alpha \|w\|^2$$

Hyperparameter

Parameter

# Trainieren von Machine Learning Modellen



# Anpassungsgüte $R^2$

Dimensionslose Grösse um die Güte eines Fits zu charakterisieren

- 0: Keine Erklärung
- 1: Perfekter fit



## Definition

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$
$$SS_{res} = \sum_{i=1}^n (y_i - f(x_i, w))^2$$
$$SS_{tot} = \sum_{i=1}^n (y_i - \bar{y})^2$$

```
from sklearn.metrics import r2_score
...
y_pred = model.predict(x)
r2 = r2_score(y, y_pred)
```

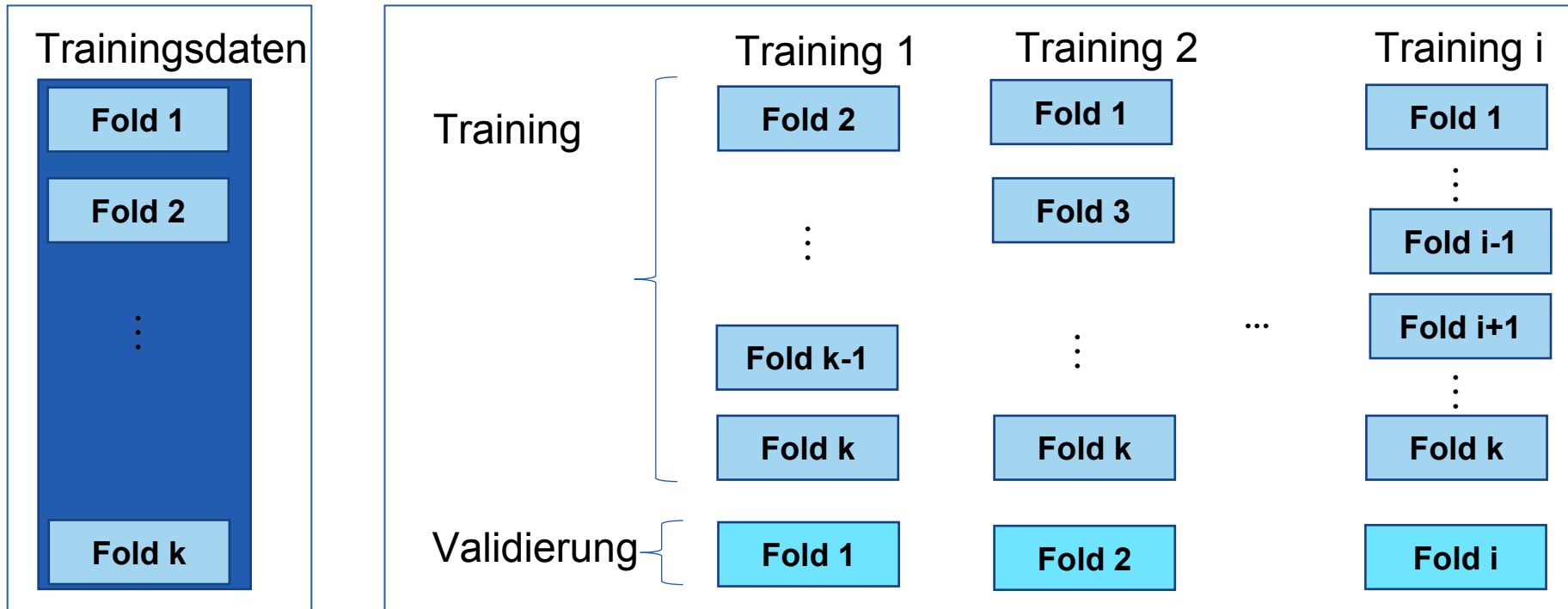
0.0: Keine Erklärung

1.0: Perfekter Fit

# Crossvalidation (Kreuzvalidierung)

Idee: Teil der Trainingsdaten wird für Validierung benutzt

- Nützlich um overfitting zu diagnostizieren

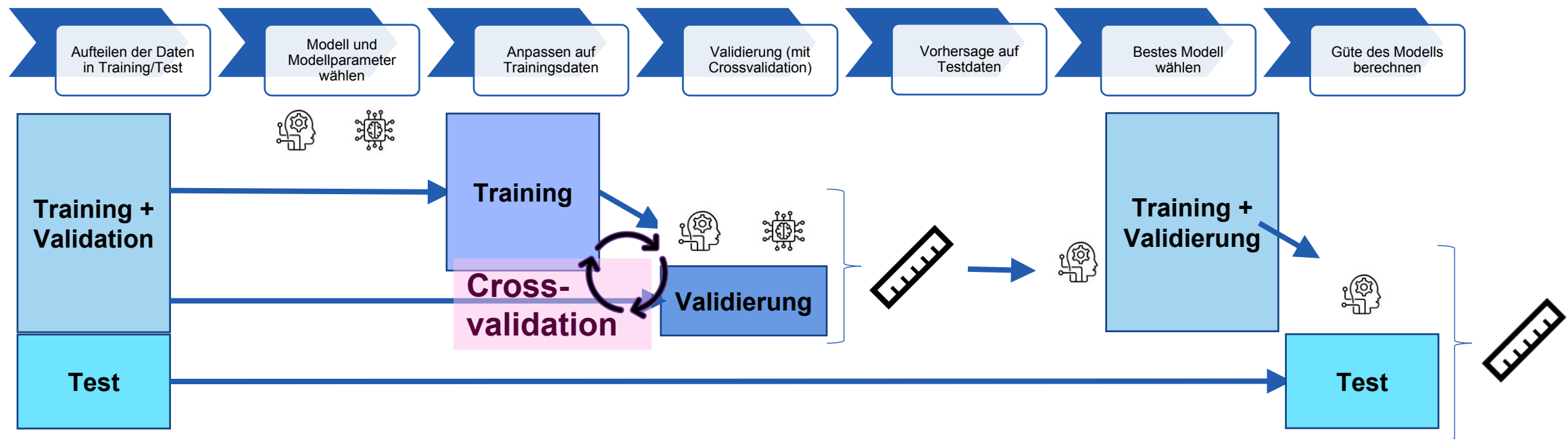


```
from sklearn.model_selection import cross_val_score
model_poly = LinearRegression()
cross_val_score(model, X, y, cv=5, scoring='r2')
```

# Finden von Hyperparameter mittels GridSearch (Modell-Selektion)

- Definiere mögliche Werte für Hyperparameter
  - Trainiere für jeden Wert ein Modell
  - Validiere Modell mittels Crossvalidation
- Wähle bestes Modell (gemäss Metrik)

```
from sklearn.model_selection import GridSearchCV
p_grid = {'alpha': [0, 0.01, 0.1, 1, 10]}
ridge = Ridge()
gs =
GridSearchCV(ridge, p_grid, cv=5, scoring='r2')
gs.fit(X, y)
```



# Trainieren von Maschine Learning Modellen



12-lecture-notebook.ipynb

# Zusammenfassung

## (Lineare) Regression

- Lineare Regression ist einfaches parametrisches Machine-learning Modell
- Interpretation von Polynomieller Regression: Daten werden transformiert, danach wird lineare Regression angewendet.
  - Erlaubt es komplexere Zusammenhänge in Daten zu erklären

**Overfitting:** Modell passt sich an Rauschen an statt Muster in Daten zu lernen

- Kann durch anwenden auf separatem Validierungsdatensatz erkannt werden
- Abhilfe: Einschränken der Modellkomplexität
  - Bei Polynomielle Regression, Polynomgrad kleiner wählen
  - Ridge regression: Bestraft grosse Koeffizienten

## Training und Evaluation eines Modells

- *Modell wird auf Datensatz trainiert (gefitted).*
- *Güte der Vorhersagen werden auf **separatem** Test Datensatz evaluiert*